Logistic Regression and Gradient Descent

By Zach Wood-Doughty;
adapted from a similar writeup by Chris Piech

April 19, 2025: v2.0

1 Introduction and notation

This note will recap the Logistic Regression model and derive the gradient update for Negative Log-Likelihood loss.

Suppose we want to consider the supervised learning task of classifying whether an animal is a rabbit. We'll represent our dataset of N examples as $\{X_i, y_i\}_{i=1}^N$. X_i is a vector of shape (1, D), where $X_{i,j}$ is the value of the j feature for animal i. D is the number of features (how many legs, has fur, etc.) that describe each animal. y_i is the label for the corresponding animal, which we will assume is either 1 (if rabbit) or 0 (if not). We'll assume there exists some (noisy) target function $f : X \to y$ such that $y_i = f(X_i)$, and our goal is to learn a hypothesis h such that $h(X) \approx f(X)$. We'll denote the parameters of our model as $w = \langle w_0, w_1, \ldots w_D \rangle$. Our hypothesis $h(X)$ is uniquely determined these parameters, and our goal is to learn these parameters from the data such that $h(X)$ is as close as possible to $f(X)$.

Unlike linear regression (ordinary least squares), this approach will not have a closed-form solution. We will have to use gradient descent to choose our parameters w.

2 The Logistic Regression model

Despite the name, this is a model for classification. The form of the model is essentially linear regression squeezed into a logistic function, which is where the name comes from.

Recall that linear regression's hypothesis looks like:

$$h(X_i) = X_i \cdot w \tag{1}$$
$$= 1 \cdot w_0 + X_{i,1} \cdot w_1 + \ldots + X_{i,D} \cdot w_D \tag{2}$$

As with linear regression, we will assume in our notation that the X input has been amended to contain an intercept term $X_{i,0} = 1$. So Equation (2) is a more verbose version of (1), but they are equivalent. For this document, assume that the 'original' X before the intercept was of shape (N, D), and after adding an intercept it is now $(N, D + 1)$. Our parameters w are thus of shape $(D + 1, 1)$.

The logistic function[1] is defined as:

$$\sigma(z) = \frac{1}{1 + \exp(-z)} \tag{3}$$

Logistic regression just puts equation (1) inside of (3), producing:

[1]The logistic function is sometimes called 'the sigmoid function' though 'sigmoid' can more broadly refer to any number of S-shaped curves.

$$h(X_i) = \frac{1}{1 + \exp(-X_i \cdot w)} \tag{4}$$

You should be able to see with a little work that $h(X_i)$ will always output a number between 0 and 1. This can be interpreted as the probability that example i takes the label 1. To actually convert that into a classification of 'rabbit' or 'not rabbit', we will just threshold at 0.5. So for computing accuracy, precision, or other binary metrics of performance, we will predict 1 if $h(X_i) > 0.5$ and otherwise predict 0.

3 Binary cross-entropy loss

We will see in lecture that mean-squared error, while it works for linear regression, is not a good choice for classification. Instead, we will use Binary Cross-entropy to compare the probabilities output by $h(X_i)$ against the correct label y_i. We will assume that $y_i \in \{0, 1\}$, though this can be extended to multi-class classification without too much additional work.

Binary cross-entropy loss for a single example (X_i, y_i) is defined as:

$$L(X_i, y_i; w) = -y_i \log(h(X_i)) - (1 - y_i) \log(1 - h(X_i)) \tag{5}$$

To use gradient descent to minimize this loss, we first need to compute the derivative of this loss with respect to our parameters w. We'll use chain rule, first noting the following partial derivatives:

$$\frac{\partial}{\partial w} X_i \cdot w = X_i \tag{6}$$

$$\frac{\partial}{\partial z} \sigma(z) = \sigma(z)(1 - \sigma(z)) \tag{7}$$

$$\frac{\partial}{\partial v} \log(v) = \frac{1}{v} \tag{8}$$

Putting these together with chain rule,[2] we get

$$\begin{aligned}
&\frac{\partial}{\partial w} L(X_i, y_i; w) \\
&\quad = \frac{\partial}{\partial w} - y_i \log(h(X_i)) - (1 - y_i) \log(1 - h(X_i)) \\
&\quad = \left(-\frac{y}{h(X_i)} + \frac{1 - y}{1 - h(X_i)} \right) \cdot h(X_i)(1 - h(X_i)) \cdot X_i \\
&\quad = (h(X_i) - y) X_i
\end{aligned} \tag{9}$$

[2] See this post or the handout by Chris Piech for helpful walkthroughs.

If we want to compute this for all data examples at once, we get:

$$\frac{\partial}{\partial w} L(\mathbf{X}, \mathbf{y}; w) = (h(\mathbf{X}) - \mathbf{y})\mathbf{X} \tag{10}$$

You can interpret this as saying that we are predicted a score or probability for each example, subtracting off by the true label, and then weighting each row of the dataset by how wrong we were on that row's prediction.

4 Gradient Descent

To implement one iteration of gradient descent for Logistic Regression in the `update_weights` function within `logistic_regression.py`, you will want to rewrite equation (10) in `numpy` code. As written above, that equation should produce an array of shape $(N, D + 1)$, where N is the number of examples and D is the number of features (not including the intercept). You will take the average of those N values to get a combined gradient update vector for your $D + 1$ weights, and then subtract that from your previous weights, multiplied by the learning rate. You may find it easiest to do this with `np.mean`. However, to write this in math notation, we will define:

$$\mathbf{1}_N = \begin{bmatrix} 1 \\ 1 \\ \dots \\ 1 \end{bmatrix} \in \mathbb{R}^{(N,1)} \tag{11}$$

Then, using η to denote the learning rate, our gradient update can be written as:

$$w_{\text{new}} = w_{\text{old}} - \frac{\eta}{N} \big[(h(\mathbf{X}) - \mathbf{y})\mathbf{X}\big]^\top \times \mathbf{1}_N \tag{12}$$

5 Additional Reading

If you want to better understand Logistic Regression, you can check out Section (1.4.6) and Chapter 8 of the Murphy Textbook, or Chapter 4 of the Bishop textbook. There are many other sources that cover it as well.